МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ «ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра математического обеспечения и применения ЭВМ

ОТЧЕТ

по лабораторной работе №4
по дисциплине «Построение и анализ алгоритмов»
Тема: Поиск подстроки в строке

Студент гр. 4345 Лунченко Н.С.
Преподаватель Жангиров Т.Р.

Санкт-Петербург
2026

Цель работы

Изучить и реализовать алгоритм Кнута-Морриса-Пратта. Применить его к решению задач поиска подстроки в строке и проверки, является ли одна строка циклическим сдвигом другой.

Задание

Часть 1. Реализуйте алгоритм КМП и с его помощью для заданных шаблона P ($|P| \leq 25000$) и текста T ($|T| \leq 500000$) найдите все вхождения P в T.

Ввод: строка P (щаблон), строка T (текст).

Вывод: индексы начал вхождений P в T, разделенные запятой; если P не входит в T, то вывести -1.

Пример

Ввод

```
ab
abab
```

Вывод

```
0,2
```

Часть 2. Заданы две строки A ($|A| \leq 5000000$) и B ($|B| \leq 5000000$).Определить, является ли A циклическим сдвигом B (это значит, что A и B имеют одинаковую длину и A состоит из суффикса B, склеенного с префиксом B). Например, defabc является циклическим сдвигом abcdef.

Вход: первая строка A, вторая строка B

Выход: если A является циклическим сдвигом B, то индекс начала строки B в A; иначе -1. Если возможно несколько сдвигов, вывести первый индекс.

Пример

Ввод

```
defabc
abcdef
```

Вывод

```
3
```

Выполнение работы

В ходе работы был реализован алгоритм Кнута-Морриса-Пратта, который по входным строкам pattern, text находит все индексы начал вхождений pattern в text за линейное время - функция kmp_search(pattern, text). Этот алгоритм использует дополнительную информацию в виде префикс-функции строки pattern, для нахождения которой также реализован отдельный алгоритм в виде функции build_prefix_function(s), возвращающей массив - префикс-функцию строки s.

Для решения конкретных задач реализованы функции solveKmp() и solveCyclicShift(), которые, используя алгоритм КМП, решают задачи соответственно: ввода паттерна и строки и нахождения списка индексов вхождений в строку, затем вывода этого списка; ввода двух строк и определения, является ли вторая строка циклическим сдвигом первой, с выводом индекса начала сдвига или -1.

Задача 1. Поиск всех вхождений подстроки в строку

Описание функции kmp_search(pattern, text)

На вход подаются строки pattern (образец) и text (текст). Если pattern пустая, функция возвращает пустой массив [].

Этап 1. Вычисление префикс-функции для pattern

Вычисляется длина m строки pattern. Выделяется и инициализируется нулями массив pi []int длины m под значения префикс-функции.

Переменные i, j инициализируются как i = 1, j = 0. Запускается цикл с условием i < m:

Если символы на позициях i, j (pattern[i] и pattern[j]) совпадают, то значение pi[i] принимается равным j + 1, после этого i и j увеличиваются на 1.

При несовпадении символов: пока j > 0 и pattern[i] != pattern[j], выполняется откат j = pi[j - 1]. После выхода из while, если pattern[i] == pattern[j], то j увеличивается на 1, затем pi[i] = j, и i увеличивается на 1.

По завершении цикла массив pi содержит префикс-функцию строки pattern.

Этап 2. Поиск всех вхождений pattern в text

Инициализируется пустой массив result []int для хранения индексов найденных вхождений. Переменная j = 0 используется для отслеживания текущей длины совпавшего префикса образца.

Запускается цикл по i от 0 до len(text) - 1:

Пока j > 0 и text[i] != pattern[j], выполняется откат j = pi[j - 1].

Если text[i] == pattern[j], то j увеличивается на 1.

Если j == m (найдено полное совпадение), то индекс начала вхождения i - m + 1 добавляется в массив result. Затем выполняется j = pi[j - 1] для продолжения поиска с учётом перекрывающихся вхождений.

По завершении цикла функция возвращает массив result.

Задача 2. Проверка, является ли одна строка циклическим сдвигом другой

Описание функции build_prefix_function(s string) []int

(Аналогично описанной в задаче 1 — реализация полностью совпадает.)

Описание функции kmp_search(pattern string, text string) int

На вход подаются строки pattern (образец) и text (текст). Если pattern пустая, функция возвращает 0 (пустая строка считается найденной в начале текста).

Вычисляется длина m строки pattern. С помощью функции build_prefix_function(pattern) получается массив префикс-функции pi для образца.

Переменная j = 0 используется для отслеживания текущей длины совпавшего префикса образца.

Запускается цикл по i от 0 до len(text)-1:

Пока j > 0 и text[i] != pattern[j], выполняется откат j = pi[j - 1].

Если text[i] == pattern[j], то j увеличивается на 1.

Если j == m (найдено полное совпадение), функция немедленно возвращает i - m + 1 (индекс первого вхождения).

Если цикл завершён без нахождения совпадения, функция возвращает -1.

Описание функции find_cyclic_shift(A string, B string) int

На вход подаются две строки A и B. Если длины строк не равны, функция возвращает -1, так как циклический сдвиг возможен только при равной длине.

Если длина строк равна 0, функция возвращает 0 (две пустые строки считаются циклическим сдвигом).

Создаётся строка double_B = B + B (сдвоенная строка, содержащая все возможные циклические сдвиги B). Вызывается функция kmp_search_first_occurrence(A, double_B), результат сохраняется в переменную pos.

Если pos != -1 и pos < len(A) (то есть найденное вхождение не выходит за пределы исходной длины A), функция возвращает pos - индекс, начиная с которого B совпадает с циклическим сдвигом A. Иначе возвращается -1 (строка B не является циклическим сдвигом A).

Сложность алгоритма

Временная сложность: $O(N_1 + N_2)$. На первом этапе за время $O(N_1)$ строится массив префикс-функции для образца. Затем основной цикл поиска вхождений выполняется за $O(N_2)$. Суммируя эти две составляющие, получаем общую линейную асимптотику.

Пространственная сложность: $O(N_1 + N_2)$. Требуется выделить память под массив префикс-функции, размер которого зависит от длины образца и составляет $O(N_1)$. Кроме того, для хранения результата - списка индексов всех вхождений подстроки - в худшем случае может потребоваться $O(N_2)$ памяти (когда образец найден на каждой позиции).

Таким образом, итоговая оценка памяти в худшем случае составляет $O(N_1 + N_2)$.

Исходный код программы см. в приложении А.

Результаты тестирования функции, вычисляющей все индексы вхождений подстроки в строку см. в Таблице 1.

Результаты тестирования функции, проверяющей по входным строкам A и B, является ли A циклическим сдвигом B см. в Таблице 2.

Таблица 1 – Результаты тестирования КМП

№	Ввод	Вывод	Комментарий
1	123 123456123	0,6	Два неперекрывающихся вхождения
2	aaa aaaaaa	0,1,2,3	Перекрытия из повторяющегося шаблона
3	ASD VC	-1	Нет искомой подстроки
4	zxc zxc	0	Строка = шаблон
5	shablon str	-1	Шаблон больше строки

Таблица 2 – Результаты тестирования нахождения циклического сдвига

№	Ввод	Вывод	Комментарий
1	defabc abcdef	3	Корректный сдвиг
2	abc bdef	-1	Разные длины строк
3	abd efc	-1	Разные строки
4	ababab bababa	1	Строка может быть получена разными сдвигами

Вывод

Реализован алгоритм Кнута-Морриса-Пратта. Дана асимптотическая оценка сложности его реализации по времени: $O(N_1 + N_2)$, памяти: $O(N_1 + N_2)$, где N_1 - длина искомого шаблона, N_2 - длина строки, в которой осуществляется его поиск. Описано применение данного алгоритма к конкретным задачам: поиска всех индексов начал вхождений шаблона в строку и проверки, является ли одна строка циклическим сдвигом друго

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл Kmp.py

```
def kmp_search(pattern, text):
    if not pattern:
        return []

    m = len(pattern)
    pi = [0] * m

    j = 0
    for i in range(1, m):
        while j > 0 and pattern[i] != pattern[j]:
            j = pi[j - 1]
        if pattern[i] == pattern[j]:
            j += 1
        pi[i] = j

    result = []
    j = 0

    for i in range(len(text)):
        while j > 0 and text[i] != pattern[j]:
            j = pi[j - 1]

        if text[i] == pattern[j]:
            j += 1

        if j == m:
            result.append(i - m + 1)
            j = pi[j - 1]

    return result


pattern = input()
text = input()

occurrences = kmp_search(pattern, text)

if occurrences:
    print(','.join(map(str, occurrences)))
else:
    print(-1)
```

Файл Kmp_cyclic.py

```
def build_prefix_function(pattern):
    m = len(pattern)
    pi = [0] * m

    j = 0
    for i in range(1, m):
        while j > 0 and pattern[i] != pattern[j]:
```

```
            j = pi[j - 1]
        if pattern[i] == pattern[j]:
            j += 1
        pi[i] = j
    return pi


def kmp_search(pattern, text):
    if not pattern or not text:
        return []

    pi = build_prefix_function(pattern)
    result = []
    j = 0

    for i in range(len(text)):
        while j > 0 and text[i] != pattern[j]:
            j = pi[j - 1]

        if text[i] == pattern[j]:
            j += 1

        if j == len(pattern):
            result.append(i - len(pattern) + 1)
            j = pi[j - 1]

    return result


def find_cyclic_shift(A, B):
    if len(A) != len(B):
        return -1

    n = len(A)

    if n == 0:
        return 0
    double_B = B + B
    occurrences = kmp_search(A, double_B)

    for pos in occurrences:
        if pos < n:
            return pos

    return -1


A = input()
B = input()

result = find_cyclic_shift(A, B)
print(result)
```